

Blooket | Question Set - Answer Key

Name _____

Date _____

Class _____

Node Quiz

1. What is Node.js?

- a) A programming language
- b) A JavaScript runtime environment ✓**
- c) A JavaScript framework
- d) A database

2. Node.js is built on which JavaScript engine?

- a) SpiderMonkey
- b) V7
- c) V8 ✓**
- d) JavaScript Core

3. What does V8 do with JavaScript?

- a) Converts JavaScript into HTML
- b) Converts JavaScript into machine-level instructions ✓**
- c) Converts JavaScript into CSS
- d) Stores JavaScript files

4. What does REPL stand for?

- a) Read-Eval-Print Loop ✓**
- b) Run-Execute-Program-Load
- c) Read-Execute-Print-List
- d) Run-Eval-Process-Loop

5. How can you start the Node.js REPL?

- a) Type npm in the terminal
- b) Type node in the terminal ✓**
- c) Type repl in the terminal
- d) Type node-repl in the terminal

6. Which execution context is created when a JavaScript file starts running?

- a) Function Execution Context
- b) Global Execution Context ✓**
- c) Module Execution Context
- d) Node Execution Context

7. Which queue has higher priority?

- a) Callback Queue
- b) Microtask Queue ✓**
- c) Task Queue
- d) Macrotask Queue

8. Which of these normally goes into the Callback/Macrotask Queue?

- a) Promise.then()
- b) queueMicrotask()
- c) setTimeout() ✓**
- d) async/await

9. Which of these normally goes into the Microtask Queue?

- a) setTimeout()
- b) setInterval()
- c) Promise.then() ✓**
- d) All of the above

10. What does the Event Loop do?

- a) Creates JavaScript variables
- b) Checks whether the Call Stack is empty and manages queued tasks ✓**
- c) Creates callback queue
- d) Create loops

11. What is the main purpose of the Call Stack?

- a) Store database records
- b) Executes current code ✓**
- c) Executes multiple codes at the same time
- d) Manages event loop

12. Who handles timer functions like setTimeout() ?

- a) Call Stack
- b) Web APIs ✓**
- c) Event Loop
- d) GEC

13. Which keyword is used to import a module in CommonJS?

- a) import()
- b) include()
- c) require() ✓**
- d) use()

14. Which is used to export values in CommonJS?

- a) export
- b) module.exports ✓**
- c) exports.module
- d) require.exports

15. Which keywords are associated with ES Modules(modern JS module)?

- a) require and module.exports
- b) import and export ✓**
- c) include and export
- d) load and require

16. Which method is used to delete a file?

- a) **fs.unlink()** ✓
- b) fs.delete()
- c) fs.link()
- d) fs.remove()

17. Which method should you use when you want to add content without replacing e content?

- a) fs.writeFile()
- b) **fs.appendFile()** ✓
- c) fs.existsFile()
- d) fs.updateFile()

18. Which method creates a directory?

- a) fs.createFolder()
- b) **fs.mkdir()** ✓
- c) fs.newDir()
- d) fs.folder()

19. What is the purpose of Node.js path module?

- a) To create databases
- b) **To work with file and folder paths** ✓
- c) To execute JavaScript
- d) To manage npm packages

20. Output?

```
const path = require("path");
const filePath = path.join("Masai", "Chitkara", "data.txt");
console.log(path.basename(filePath));
```

- a) **data.txt** ✓
- b) .txt
- c) /Masai/Chitkara
- d) /Masai/Chitkara/data.txt

21. What does path.resolve() returns?

- a) **Absolute path(complete path)** ✓
- b) Relative path
- c) Resolves a path
- d) Creates directory

22. Output?

```
console.log("Start");

setTimeout(() => {
  console.log("Timeout");
}, 0);

Promise.resolve().then(() => {
  console.log("Promise");
});

console.log("End");
```

- a) Start, Timeout, Promise, End
- b) Start, Promise, Timeout, End
- c) Start, End, Timeout, Promise
- d) Start, End, Promise, Timeout ✓**

23. Output?

```
setTimeout(() => {
  console.log("Timeout 1");
}, 3000);

setTimeout(() => {
  console.log("Timeout 2");
}, 1000);

Promise.resolve().then(() => {
  console.log("Promise");
});
```

- a) Timeout 1, Timeout 2, Promise
- b) Promise, Timeout 1, Timeout 2
- c) Promise, Timeout 2, Timeout 1 ✓**
- d) Timeout2, Timeout1, Promise

24. Output?

```
setTimeout(() => {
  console.log("1");

  Promise.resolve().then(() => {
    console.log("2");
  });
}, 0);

Promise.resolve().then(() => {
  console.log("3");
});
```

- a) 1,2,3
- b) 1,3,2
- c) 3,2,1
- d) 3,1,2 ✓**

25. Node.js executes JavaScript using:

- a) Multi JavaScript threads
- b) Single main JavaScript thread ✓**
- c) No thread
- d) Database thread